

Practical-02

Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

OUTPUT:

```
v_santosh@DESKTOP-E34M7RI:~$ mkdir file_copy
v_santosh@DESKTOP-E34M7RI:~$ mkdir Practical_02
v_santosh@DESKTOP-E34M7RI:~$ cd Practical_02
v_santosh@DESKTOP-E34M7RI:~/Practical_02$ pwd
/home/v_santosh/Practical_02
v_santosh@DESKTOP-E34M7RI:~/Practical_02$ nano filecopy.c
v_santosh@DESKTOP-E34M7RI:~/Practical_02$ echo "Hello, this is a sample file." > sample.txt
v_santosh@DESKTOP-E34M7RI:~/Practical_02$ cat sample.txt
Hello, this is a sample file.
v_santosh@DESKTOP-E34M7RI:~/Practical_02$ gcc filecopy.c -o filecopy
v_santosh@DESKTOP-E34M7RI:~/Practical_02$ ls
filecopy filecopy.c sample.txt
v_santosh@DESKTOP-E34M7RI:~/Practical_02$ ./filecopy
File copied successfully.
```

```

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int source, destination;
    char buffer[1024];
    ssize_t bytes;

    source = open("sample.txt", O_RDONLY);

    if (source < 0)
    {
        perror("Error opening source file");
        return 1;
    }

    destination = open("copy.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination < 0)
    {
        perror("Error opening destination file");
        close(source);
        return 1;
    }

    while ((bytes = read(source, buffer, sizeof(buffer))) > 0)
    {
        if (write(destination, buffer, bytes) != bytes)
        {
            perror("Error writing file");
            close(source);
            close(destination);
            return 1;
        }
    }

    if (bytes < 0)
        perror("Error reading file");

    close(source);
    close(destination);

    printf("File copied successfully.\n");

    return 0;
}

```